

Pilha e Fila (stack e queue)

pilha-e-fila.md

Sumário

Pilha e Fila (stack e queue)	4
1. Por que existem estruturas tão simples?	4
Parte 1: Pilha (stack)	5
2. A ideia	5
3. As operações	5
Esvaziando uma pilha	6
Percorrendo uma pilha	6
4. Onde a pilha aparece	7
4.1 Inverter uma sequência	7
4.2 “Desfazer” (Ctrl+Z)	7
4.3 Coisas que “abrem” e “fecham”	7
4.4 Chamadas de função e recursão	7
5. stack ou vector?	7
6. Aplicação: o problema 2406 (Expressões)	8
O enunciado, em resumo	8
A dica	8
Parte 2: Fila (queue)	9
7. A ideia	9
8. As operações	9
9. Onde a fila aparece	10
9.1 Simular uma fila de verdade	10
9.2 Rodadas / turnos	10
9.3 Busca em largura (BFS)	10
10. Aplicação: o problema 2376 (Copa do Mundo)	10
O enunciado, em resumo	10
A dica	10
Parte 3: A prima deque	11

11. deque: fila de duas pontas	11
12. Erros comuns	12

Pilha e Fila (stack e queue)

Este material continua os anteriores de Introdução ao C++, `vector`, Busca Binária e Ordenação e Dois Ponteiros. Até aqui, o `vector` foi a nossa única estrutura para guardar dados. Agora vamos conhecer duas estruturas que são mais “limitadas” que o `vector` de propósito, e é justamente essa limitação que torna alguns problemas muito mais simples de resolver:

1. A **pilha** (stack): o último que entra é o primeiro que sai;
2. A **fila** (queue): o primeiro que entra é o primeiro que sai.

No fim, veremos também a prima das duas, a **fila de duas pontas** (deque), que junta o que a pilha e a fila fazem em uma única estrutura.

1. Por que existem estruturas tão simples?

O `vector` deixa você fazer quase tudo: acessar qualquer posição, inserir e remover em qualquer lugar, ordenar, percorrer. Então por que usar algo que faz *menos*?

Porque em muitos problemas a única coisa que importa é **a ordem em que as coisas entram e saem**. Pense em situações do dia a dia:

- Uma **pilha de pratos**: você coloca um prato em cima e, quando vai pegar, pega o de cima. O prato lá do fundo só sai depois de todos os outros.
- Uma **fila de banco**: quem chegou primeiro é atendido primeiro. Quem chega agora vai para o fim da fila.

Quando um problema tem esse formato, usar uma pilha ou uma fila deixa o código mais **curto**, mais **fácil de ler** e, principalmente, mais **difícil de errar**, porque a estrutura não deixa você fazer nada fora da regra.

Os dois nomes que você vai ouvir o tempo todo:

Estrutura	Regra	Sigla
Pilha (stack)	Último a entrar, primeiro a sair	LIFO (<i>Last In, First Out</i>)
Fila (queue)	Primeiro a entrar, primeiro a sair	FIFO (<i>First In, First Out</i>)

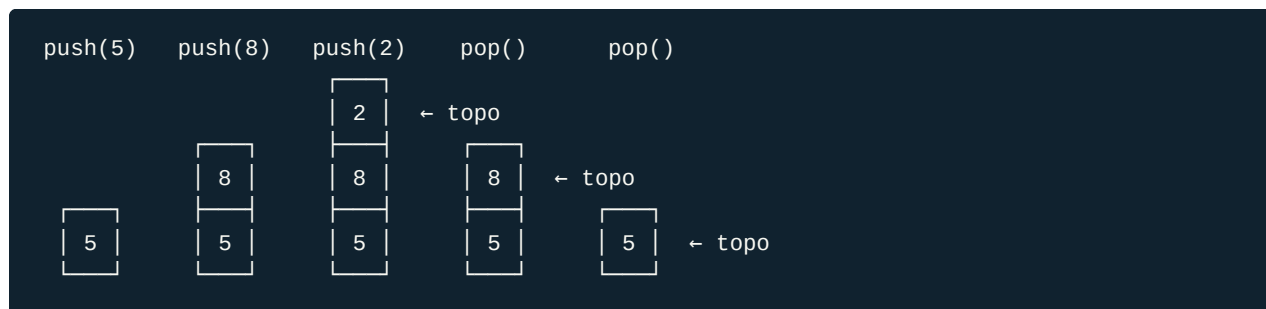
NOTA

As duas vêm dos cabeçalhos `<stack>` e `<queue>`, mas como usamos `#include <bits/stdc++.h>`, já estão incluídas.

Parte 1: Pilha (stack)

2. A ideia

Uma pilha só tem **uma porta**: o **topo**. Tudo entra pelo topo e tudo sai pelo topo.



Entraram na ordem 5, 8, 2 e saem na ordem 2, 8, 5: a pilha **inverte** a ordem. Guarde isso, porque é o “superpoder” dela.

3. As operações

```
#include <bits/stdc++.h>
using namespace std;

int main () {
    stack<int> p;           // pilha de inteiros, começa vazia

    p.push(5);             // coloca no topo
    p.push(8);
    p.push(2);

    cout << p.top() << '\n'; // 2 (olha o topo, NÃO remove)
    p.pop();               // remove o topo (o 2)
    cout << p.top() << '\n'; // 8

    cout << p.size() << '\n'; // 2 (quantos elementos tem)

    if (p.empty()) cout << "vazia\n";
    else           cout << "tem coisa\n"; // tem coisa
}
```

cpp

Função	O que faz
<code>p.push(x)</code>	Coloca <code>x</code> no topo
<code>p.pop()</code>	Remove o elemento do topo (não devolve ele)
<code>p.top()</code>	Devolve o elemento do topo, sem remover

Função	O que faz
<code>p.size()</code>	Quantos elementos há
<code>p.empty()</code>	<code>true</code> se está vazia

Todas as operações são instantâneas, não importa o tamanho da pilha. É por isso que ela aguenta tranquilamente entradas com centenas de milhares de elementos.

ATENÇÃO

`pop()` **não devolve nada**. Se você quer usar o valor que vai sair, primeiro leia com `top()`, depois remova com `pop()`:

```
cpp int x = p.top(); p.pop();
```

Esse é um erro muito comum de quem vem de outras linguagens, em que o `pop` devolve o elemento.

CUIDADO

Chamar `top()` ou `pop()` em uma pilha **vazia** não gera erro de compilação nem uma mensagem bonitinha: o programa simplesmente faz algo imprevisível (geralmente trava com *Runtime Error*). **Sempre** confira `empty()` antes, a não ser que você tenha certeza absoluta de que há algo lá.

Esvaziando uma pilha

Não existe `p.clear()`. Duas formas de esvaziar:

```
while (!p.empty()) p.pop();    // forma 1: remove um a um

p = stack<int>();              // forma 2: substitui por uma pilha nova
```

cpp

Percorrendo uma pilha

A pilha não tem `p[i]` nem `begin()/end()`. Se você precisa “ver todos os elementos”, a única forma é ir tirando do topo, o que **destrói** a pilha:

```
while (!p.empty()) {
    cout << p.top() << ' ';
    p.pop();
}
```

cpp

Se isso incomoda no seu problema, é sinal de que talvez você queira um `vector` em vez de uma `stack` (veja a seção 5).

4. Onde a pilha aparece

4.1 Inverter uma sequência

Como a pilha devolve tudo ao contrário, inverter é automático:

```
string s = "OBI";
stack<char> p;
for (char c : s) p.push(c);

while (!p.empty()) {
    cout << p.top();    // IBO
    p.pop();
}
```

cpp

(Para strings e vetores, `reverse(s.begin(), s.end())` faz o mesmo em uma linha. O exemplo serve para ver a inversão acontecendo.)

4.2 “Desfazer” (Ctrl+Z)

Cada ação vai para a pilha. Desfazer é tirar do topo: você sempre desfaz a ação **mais recente** primeiro. Editores de texto, navegadores (botão “voltar”) e jogos funcionam assim.

4.3 Coisas que “abrem” e “fecham”

Esse é o uso clássico em prova: parênteses, colchetes, chaves, tags de HTML, blocos de código. Toda vez que algo **abre**, guardamos na pilha; toda vez que algo **fecha**, ele precisa combinar com o que está no **topo**, o último que abriu. A seção 6 traz um problema para você resolver com isso.

4.4 Chamadas de função e recursão

Quando uma função chama outra, o computador guarda “onde parou” em uma pilha (a famosa *call stack*). Quando a função de dentro termina, volta para a mais recente. Por isso a recursão e a pilha são tão parecidas, e por isso uma recursão sem fim dá *stack overflow*.

5. `stack` ou `vector`?

Um detalhe que surpreende: o `vector` **também** funciona como pilha!

Na <code>stack</code>	No <code>vector</code>
<code>p.push(x)</code>	<code>v.push_back(x)</code>
<code>p.pop()</code>	<code>v.pop_back()</code>

Na <code>stack</code>	No <code>vector</code>
<code>p.top()</code>	<code>v.back()</code>
<code>p.empty()</code>	<code>v.empty()</code>

E o `vector` ainda deixa olhar qualquer posição com `v[i]` e percorrer com `for`.

Então por que usar `stack`? Por **clareza**. Quando alguém lê `stack<int> p;`, entende na hora: “isso aqui só entra e sai pelo topo”. Com `vector`, essa intenção fica escondida. Em prova, use a que você achar mais confortável; as duas têm o mesmo custo.

Regra prática: se você **só** precisa do topo, `stack`. Se em algum momento precisa olhar o meio ou percorrer sem destruir, `vector`.

6. Aplicação: o problema 2406 (*Expressões*)

O enunciado, em resumo

São dadas `T` cadeias formadas apenas pelos caracteres `(, ), [, ], { e }`, cada uma com até 100.000 caracteres. Uma cadeia é **bem definida** se todo símbolo que abre é fechado pelo símbolo correspondente, na ordem certa. Para cada cadeia, imprima `S` se ela é bem definida e `N` caso contrário.

```
( )      → S
([{}])   → S
{ } ( ) [ ] → S
[ ]      → N      fecha com o tipo errado
}{       → N      fecha antes de abrir
{ [ ]    → N      abriu e nunca fechou
```

A dica

Olhe para `( [{}])`. Quando aparece o `}`, qual símbolo ele tem que fechar? O `{`, que foi o **último** a abrir. Depois, o `]` tem que fechar o `[`, que é o último que ainda está aberto. E assim por diante.

“O último que abriu e ainda não fechou” é exatamente o **topo de uma pilha**. Pense no que fazer quando um caractere **abre**, no que fazer quando um caractere **fecha**, e no que a pilha precisa estar “dizendo” quando a cadeia termina. O resto é com você.

Um detalhe de leitura que costuma travar todo mundo: depois de ler `T` com `cin >> t`, o `Enter` dessa linha fica sobrando na entrada. Se você for usar `getline` para ler cada cadeia, chame `cin.ignore()` antes, senão a primeira linha lida vem vazia.

Parte 2: Fila (queue)

7. A ideia

A fila tem **duas portas**: entra por uma (o **fim**, *back*) e sai pela outra (a **frente**, *front*).

```
push(5)  push(8)  push(2)                pop()        pop()
frente → [5] [8] [2] ← fim      frente → [8] [2]      frente → [2]
```

Entraram 5, 8, 2 e saem 5, 8, 2: a fila **preserva** a ordem de chegada.

8. As operações

```
#include <bits/stdc++.h>
using namespace std;

int main () {
    queue<int> f;

    f.push(5);           // entra no fim
    f.push(8);
    f.push(2);

    cout << f.front() << '\n'; // 5 (o primeiro que entrou)
    cout << f.back() << '\n';  // 2 (o último que entrou)

    f.pop();             // remove o da frente (o 5)
    cout << f.front() << '\n';  // 8

    cout << f.size() << '\n';   // 2
    if (f.empty()) cout << "vazia\n";
}
```

cpp

Função	O que faz
<code>f.push(x)</code>	Coloca <code>x</code> no fim
<code>f.pop()</code>	Remove o elemento da frente (não devolve ele)
<code>f.front()</code>	Devolve o da frente, sem remover
<code>f.back()</code>	Devolve o do fim, sem remover
<code>f.size()</code>	Quantos elementos há
<code>f.empty()</code>	true se está vazia

Os mesmos avisos da pilha valem aqui: `pop()` não devolve nada (leia com `front()` antes), e `front()/pop()` em fila vazia é comportamento indefinido; confira `empty()`.

Também não existe `clear()`; esvazie com `while (!f.empty()) f.pop();` ou `f = queue<int>();`.

9. Onde a fila aparece

9.1 Simular uma fila de verdade

Atendimento em banco, fila de impressão, pedidos de um restaurante, jogadores esperando a vez em um jogo. Sempre que o problema diz “o primeiro que chegou é o primeiro atendido” ou “quem terminou volta para o fim da fila”, é `queue`.

9.2 Rodadas / turnos

Uma variação muito comum: um elemento sai da frente, faz alguma coisa e **volta para o fim**. Exemplo: `n` crianças brincando de batata quente, ou a ordem de jogadas em um jogo de tabuleiro.

```
queue<int> f;
for (int i = 1; i <= n; i++) f.push(i);

while (f.size() > 1) {
    int atual = f.front();
    f.pop();
    // ... atual faz a sua jogada ...
    f.push(atual);           // volta para o fim da fila
}
cout << f.front();         // o único que sobrou
```

cpp

9.3 Busca em largura (BFS)

Este é o uso mais importante da fila em programação competitiva: explorar um labirinto, um mapa ou um grafo “em camadas”, visitando primeiro tudo que está a 1 passo, depois tudo a 2 passos, e assim por diante. A fila garante que ninguém “fura” a ordem. Vamos estudar BFS em um material próprio; por enquanto, basta saber que **BFS = fila**.

10. Aplicação: o problema 2376 (Copa do Mundo)

O enunciado, em resumo

Dezesseis equipes, A até P, disputam 15 jogos eliminatórios em uma tabela de “mata-mata”: jogo 1 é A × B, jogo 2 é C × D, ..., jogo 8 é O × P. O jogo 9 é entre os vencedores dos jogos 1 e 2, o jogo 10 entre os vencedores dos jogos 3 e 4, e assim por diante até a final (jogo 15). São dados os placares dos 15 jogos, em ordem, e não há empates. Imprima a letra da campeã.

A dica

Vamos olhar a ordem em que as equipes jogam:

```
Jogo 1:  A × B      Jogo 2:  C × D      ...      Jogo 8:  O × P
Jogo 9:  venc(1) × venc(2)
Jogo 10: venc(3) × venc(4)
...
```

Repare: o vencedor do jogo 1 é o **primeiro** a entrar na “lista de quem ainda está vivo”, e ele também é o **primeiro** a jogar de novo (jogo 9). O vencedor do jogo 2 é o segundo a entrar e o segundo a jogar. Quem vence primeiro, joga primeiro de novo: **FIFO**.

Se as 16 equipes começarem em uma fila, na ordem certa, quem são os dois que se enfrentam a cada jogo? E o que você faz com o vencedor? Responda essas duas perguntas e o problema praticamente se resolve sozinho; nem precisa calcular “quem enfrenta quem”.

Parte 3: A prima deque

11. deque: fila de duas pontas

O deque (*double-ended queue*, pronuncia-se “déc”) é uma fila em que você pode inserir e remover **pelas duas pontas**, e ainda acessar qualquer posição com `d[i]`, como um vector.

```
deque<int> d;

d.push_back(1);    // [1]
d.push_back(2);    // [1, 2]
d.push_front(0);   // [0, 1, 2]

cout << d.front(); // 0
cout << d.back();  // 2
cout << d[1];      // 1

d.pop_front();     // [1, 2]
d.pop_back();      // [1]
```

cpp

Função	O que faz
<code>d.push_back(x)</code> / <code>d.push_front(x)</code>	Insere no fim / na frente
<code>d.pop_back()</code> / <code>d.pop_front()</code>	Remove do fim / da frente
<code>d.back()</code> / <code>d.front()</code>	Olha o fim / a frente
<code>d[i]</code>	Acessa a posição <code>i</code>

Função	O que faz
<code>d.size()</code> , <code>d.empty()</code> , <code>d.clear()</code>	Como no <code>vector</code>

Ele também aceita `begin()/end()`, então `sort`, `reverse` e o `for (int x : d)` funcionam.

Quando usar: quando você precisa de uma fila (ou pilha) mas *também* precisa olhar o meio, ou inserir/remover nas duas pontas. Um `vector` tem `push_front`? Não: inserir na frente de um `vector` é lento, porque todos os outros elementos precisam ser deslocados uma casa para a direita. O `deque` faz isso na hora.

Na prática, `stack` e `queue` são só “cascas” em volta de um `deque` que escondem as funções que não fazem sentido para cada uma. Por isso muita gente usa direto o `deque` quando quer as duas coisas.

NOTA

Para uso comum, prefira `vector`: ele é um pouco mais rápido e tem menos pegadinhas. Só troque por `deque` quando precisar de `push_front/pop_front`.

12. Erros comuns

Nas duas (`stack` e `queue`):

- Achar que `pop()` devolve o elemento. Ele só remove. Leia com `top()/front()` antes.
- Chamar `top()`, `front()` ou `pop()` com a estrutura **vazia**. Confira `empty()`: é a causa número um de *Runtime Error* com essas estruturas.
- Procurar `clear()`. Não existe em `stack` nem em `queue`; esvazie com um `while (!x.empty()) x.pop();`.
- Tentar usar `x[i]` ou `for (auto e : x)`. Não tem. Se você precisa disso, quer um `vector` ou um `deque`.
- Declarar a estrutura **fora** do laço de casos de teste e esquecer de esvaziá-la entre um caso e outro. O lixo do caso anterior contamina o seguinte.

Confusões entre elas:

- Usar `front()` em `stack` ou `top()` em `queue`. Pilha tem `top`; fila tem `front` e `back`.
- Usar fila onde a ordem tinha que ser invertida, ou pilha onde tinha que ser preservada. Pergunte: “quem tem que sair agora: o último que entrou ou o primeiro?”